MYTHBOUND

Technical Overview: tech stack, system design and generative AI

Version 1.0 · github.com/Rakesh-Tummala/Genai_game

Contents

1. Project summary
2. Tech stack
3. System architecture
4. Client design (the game)
5. Server design
6. Generative AI in MYTHBOUND
7. Security and safety
8. Build, run and deploy
9. Testing and verification
10. Limitations and future work

1. Project summary

MYTHBOUND is a browser-based, 16-bit style creature-taming RPG. The player travels through eight mythic cities (India, Greece, Norse, Egypt, Japan, Celtic, Mesopotamia and Slavic folklore), tames creatures, battles trainers and lieutenants, and completes a trial set by each city's god.

Its signature feature is **generative AI inside gameplay**. The player can type any attack idea in battle and an AI turns it into a real, balanced move. AI also writes villager dialogue, Bestiary lore and "Mythic Form" titles for tamed creatures.

Fact	Value
Deliverable	One self-contained file, `mythbound.html` (~416 KB), plus an optional Node server
Source	~4,800 lines of JavaScript in 20 source files plus `server.js` and `build.js`
Content	10 playable characters, 8 cities, 70 creatures, 59 moves, 7 orbs, 7 items, 47 trainers, lieutenants and gods
Assets	None. All art, fonts, music and sound are generated in code at runtime
Runtime dependencies (game)	Zero. Vanilla JS, Canvas 2D and Web Audio
Runtime dependencies (server)	Node 18+, plus `@anthropic-ai/sdk` (only for the Claude provider)

> **Design principle:** the game must be fully playable offline. AI is an enhancement layer that can fail at any moment without breaking anything.

2. Tech stack

Client (the game)

Layer	Technology	Used for
Language	Vanilla JavaScript (ES2020)	All game logic. No frameworks, no bundler.
Rendering	HTML5 Canvas 2D (320×240, pixel-scaled)	World, battles, menus, effects
Graphics	Procedural pixel art in code	Creature and character sprites, tiles, buildings, backgrounds
Text	Custom bitmap font from a glyph table	All on-screen text, with no web fonts
Audio	Web Audio API	Chiptune music (pulse, triangle, saw, pluck voices), sound effects, limiter
Input	Keyboard events plus an on-screen touch pad	Desktop and mobile controls
Storage	`localStorage`	Save games and settings
Networking	`fetch` with `AbortController` timeouts	Optional calls to the AI server

Server (optional)

Layer	Technology	Used for
Runtime	Node.js 18+ (built-in `http` and `fetch`)	Serves the game and the `/api/ai` endpoint. No web framework.
AI provider 1	Anthropic Claude through `@anthropic-ai/sdk`	Highest-quality text with structured JSON output
AI provider 2	Ollama (local LLM, default `llama3.2:3b`)	Free, offline, private AI on the player's own GPU
AI provider 3	Any OpenAI-compatible API (Groq, OpenRouter, Gemini, LM Studio)	Hosted or local alternatives
Test mode	Built-in MOCK provider	Canned JSON replies, so no key or model is needed

Tooling

Tool	Purpose
`build.js` (Node)	Concatenates `src/*.js` into `shell.html` to produce the single-file game
Git and GitHub	Version control, published in four stages (v0.1 to v1.0)
Headless browser automation	Simulated full playthroughs, balance testing and screenshot capture
OfflineAudioContext	Rendering the soundtrack offline to check volume and clipping

3. System architecture

MYTHBOUND has three tiers, and only the first is required.

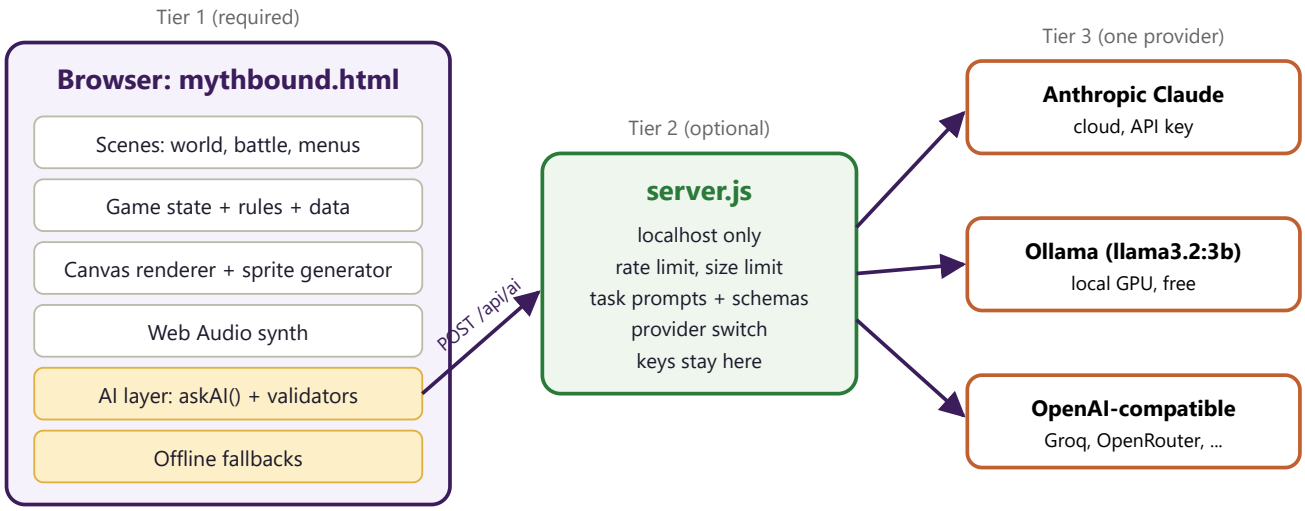


Figure 1. Three-tier architecture. The game works on its own; the server and AI provider are optional.

Key decisions

Decision	Why
Single HTML file, no assets	Runs anywhere by double-clicking; nothing to install, host or download.
No frameworks	Small size, no build chain to break, full control of the frame loop.
AI behind a local server	API keys never reach the browser, and providers can change without touching the game.
AI describes, code decides	LLM output is unpredictable, so all numbers come from deterministic, validated game rules.
Offline fallback for every AI feature	The game never waits on, or breaks because of, the network or a model.
Pluggable providers	Players can choose quality (Claude), privacy and cost (Ollama), or a hosted free tier.

4. Client design (the game)

Source layout

Files are numbered so `build.js` concatenates them in dependency order.

Files	Module	Responsibility
01_core	Core	Constants, RNG, async task and tween system, settings, AI endpoint
02_font	Font	Bitmap glyph table, text drawing and wrapping, sanitising
03_audio	Sound	Synth voices, per-city songs, sound effects, master limiter
04_input	Input	Keyboard and touch mapping, pressed/held state per frame
05_data*	Data	Types and type chart, species, moves, items, characters, trainers, cities
06_sprites*	Art	Procedural creature templates, palettes, people, tiles, buildings
07_ai	AI layer	`askAI`, JSON coercion, custom-move builder, offline interpreter
08_game	State	Game object, stats, XP, party, bag, save and load
09_ui	UI	Windows, dialog boxes, menus, the custom-attack text box
10_world*	Overworld	Maps, camera, movement, NPCs, warps, encounters, day/night
11_battle	Battle	Turn engine, damage, statuses, taming, XP, evolution, effects
12_scenes	Scenes	Title, character select, starters, Bestiary, world map, credits
13_main	Main loop	Fixed 60 Hz update, draw, canvas scaling

Scene stack and game loop

- **Fixed timestep:** `update()` runs at 60 Hz (up to 4 catch-up steps per frame), and `draw()` runs once per animation frame.
- **Scene stack:** the top scene gets input. Scenes marked opaque hide those below, so a menu can draw over the battle.
- **Async flow:** scripts like dialogue, battles and cutscenes are written as `async` functions (`await say()`, `await choose()`, `await wait(n)`) that resume on frame ticks. This keeps story logic readable and linear.

Procedural content

- **Creatures:** 7 body templates (quadruped, winged, avian, serpentine, humanoid, aquatic, spirit), each combined with palettes, patterns, accessories and a 2-frame idle animation.
- **Cities:** each culture has a theme (tile colours, trees, roof style, particles such as petals, snow, sand or blossoms) and its own buildings.
- **Music:** songs are note data played by oscillator voices with harmony lines, routed through a compressor that acts as a limiter.

Battle engine

- Speed order with priority moves, STAB (same-type bonus), critical hits and an 8-type chart (2x / 0.5x / 0x).
- Statuses (burn, poison, sleep, stun, bind), stat stages, shields, terrain, PP and EN (energy).
- Taming uses a formula based on HP, status, rarity and orb, with up to 4 shake checks.
- XP, levelling to 50, learning moves and evolution. Bench companions get a third of the XP.

5. Server design

`server.js` is a single dependency-light Node file with two jobs:

1. **Serve the game:** `GET /` returns `mythbound.html` with `AI_ENDPOINT` set to `"/api/ai"`, so the file on disk stays in offline mode.
2. **Run AI tasks:** `POST /api/ai` with `{ task, payload }` returns `{ ok, data }`.

Request pipeline

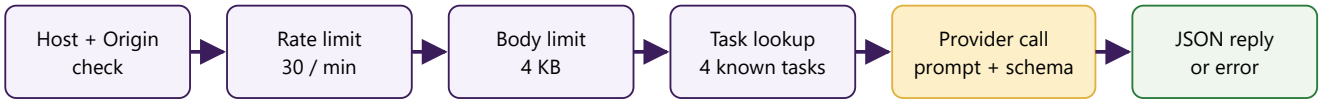


Figure 2. Every request passes the same checks before any AI provider is called.

Providers

Provider	How it is called	Structured output
`anthropic`	Official SDK, model `claude-opus-5-5`, low effort	JSON-schema output format
`ollama`	`POST /api/chat` to the local Ollama server; model kept loaded for 30 min	JSON schema passed in `format`
`openai`	`POST {base}/chat/completions` with optional bearer key	JSON mode plus field list in the prompt
MOCK	No network	Canned replies

Configuration

Variable	Default	Meaning
AI_PROVIDER	anthropic	anthropic, ollama or openai
ANTHROPIC_API_KEY	(none)	Claude key; without it, MOCK mode
AI_MODEL	per provider	Model name
OLLAMA_URL	http://127.0.0.1:11434	Ollama address
AI_BASE_URL, AI_API_KEY	(none)	OpenAI-compatible endpoint and key
AI_TIMEOUT_MS	8000	Server-side wait for Ollama / OpenAI-compatible replies
PORT, RATE_LIMIT, MOCK, ALLOW_FILE_ORIGIN	8787, 30, 0, 0	Port, requests per minute, force mock, allow file:// pages

When Ollama is selected, the server sends a warm-up request at startup (waiting up to 3 minutes), so the first in-game request is not slowed by loading the model into GPU memory.

6. Generative AI in MYTHBOUND

Generative AI is used for **creative text that makes each playthrough feel unique**. It is deliberately **not** used for anything that decides balance, progress or rewards.

The four AI tasks

Task	When	Input (payload)	Output (JSON)
`custom_attack`	Player types an idea in battle	Idea text, creature name, types, level, known moves, battle summary	`name, type, category, effects[{kind, rating 1-5}], description`
`npc_dialogue`	Talking to certain villagers	NPC name and role, city, player name, story progress	`line`
`lore`	Z on a Bestiary entry	Species, types, short lore, home city	`lore` (2-3 sentences)
`mythic_form`	After taming a creature	Species, types, lore	`epithet, up, down` (stats)

Prompt design

- **Shared rules prompt:** JSON only, family-friendly, respectful portrayal of gods and folklore, original content.
- **Per-task instructions** with strict length limits (for example name ≤ 22 characters, description ≤ 140), plain ASCII, and "rate absolute ideas low".
- **JSON schemas** for every task. Enums restrict effect kinds (15 kinds such as damage, shield, heal, burn, sleep, atk_down, terrain) and stats (hp, atk, def, spd).
- **Context, not freedom:** the model receives only the game data it needs, so its job is narrow and predictable.

The custom-attack pipeline

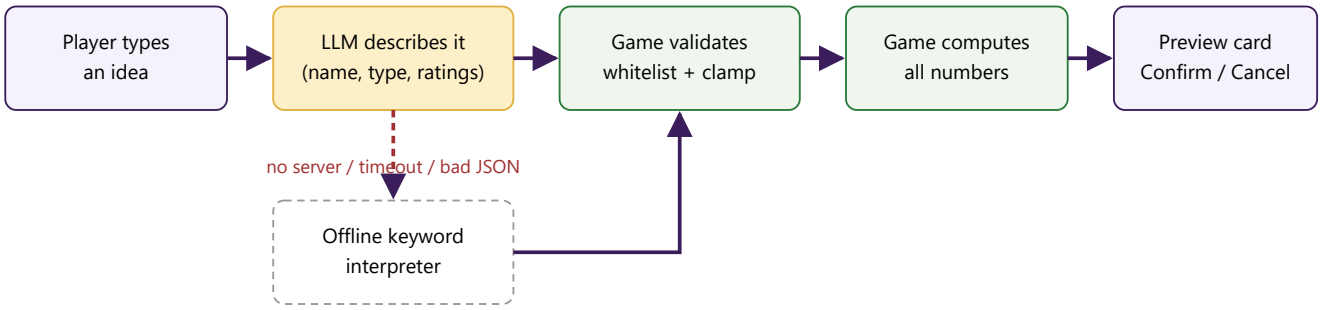


Figure 3. The AI only describes the move. The game validates it and sets every number.

What the game code enforces (in `buildCustomMove`):

- Unknown fields are ignored; ratings are clamped to 1-5; at most one primary and one secondary effect.
- The type must be one of the creature's types or Neutral.
- A **power budget** based on level and base stats scales ratings down; the preview shows a note when that happens.
- Power, accuracy, EN cost, cooldown and duration come from fixed formulas.

- Hard caps: a custom hit never deals more than 50% of the target's max HP, no healing above max, no revives, nothing outside the battle.
- 3 custom attacks per battle (4 for one character), with a cooldown and an EN cost.

Reliability: graceful degradation

Failure	What happens
No server (file opened directly)	Offline interpreter and pre-written text from the start
Server down or model unloaded	Client times out after 9 s, marks AI as "fallback", uses offline content
Invalid or partial JSON	`coerceJSON` extracts the first JSON object; if that fails, offline content
Rule-breaking output	Validator clamps or rejects fields; numbers are always game-made

Local AI with Ollama

- Model: **llama3.2:3b** (2 GB), installed to `W:\Ollama` with models in `W:\ollama\models`.
- Hardware tested: NVIDIA RTX 3050 Laptop GPU (4 GB). About 80% of the model runs on the GPU.
- Measured speed: cold load about 40-55 s (handled by the startup warm-up), then **1-2 s per request**.
- Trade-off: free and private, but simpler wording than Claude. The validator keeps gameplay identical regardless of model quality.

Example (real output from the local model)

```
Idea:   "Summon a monsoon wave that crashes down and shields me"
AI:     { "name": "Monsoon Shield", "type": "Tide", "category": "debuff",
          "effects": [ {"kind":"def_down","rating":3}, {"kind":"def_up","rating":1} ],
          "description": "A massive wave crashes down, shielding Makarin..." }
Game:   Accuracy 83%, cost 10 EN, cooldown 2 turns, ratings scaled to the Lv 8 budget
```

```
Idea:   "instantly win the battle"
AI:     rated as damage 2 + stun 2 (weak), as the rules require
Game:   capped at 50% max HP per hit, so an instant win is impossible
```

7. Security and safety

Risk	Mitigation
API key leakage	Keys are read from environment variables on the server only; the HTML never contains them.
Other sites calling the local server	Binds to 127.0.0.1; checks Host and Origin headers; CORS only for the same origin (file:// pages only if explicitly allowed).
Abuse and runaway cost	Rate limit (30 requests per minute per client), 4 KB request bodies, fixed task list, short max output tokens.
Prompt injection through the attack idea	Idea is sanitised and capped at 120 characters; the model can only fill a strict schema; the game validates every field and computes all numbers.
Unsafe or off-tone text	Family-friendly, respectful rules in every prompt; length limits; plain ASCII; the bitmap font cannot render arbitrary markup.
Cheating through AI	The AI cannot grant items, money, XP, taming or wins; damage caps and budgets are enforced in code.

8. Build, run and deploy

Build

```
node build.js        # src/*.js (sorted) -> shell.html placeholder -> mythbound.html
```

Run

Mode	Command
Offline	Open `mythbound.html` in a browser
MOCK AI	`npm run mock`
Claude	`ANTHROPIC_API_KEY=... node server.js`
Local AI (free)	`AI_PROVIDER=ollama node server.js` (PowerShell: `$env:AI_PROVIDER="ollama"; node server.js`)
OpenAI-compatible	`AI_PROVIDER=openai AI_BASE_URL=... AI_API_KEY=... AI_MODEL=... node server.js`

Then open `http://localhost:8787`. Because the game is one static file, the offline version can also be hosted on any static site (for example GitHub Pages).

9. Testing and verification

- **Automated playthroughs:** a scripted player drove the real game (menus, battles, taming, trainers, all eight gods and the credits) to check that it can be completed and to tune difficulty. Final run: all 8 seals, 9 losses in total, mostly in the early cities.
- **Balance tuning:** XP formula (`species XP × level / 4`, trainers ×1.5, bench gets a third) adjusted after the simulated runs.
- **Audio checks:** every track rendered with OfflineAudioContext and checked for clipping, silence and NaN values; loudness raised and limited.
- **AI checks:** all four tasks tested against MOCK, the Claude path (key handling and errors) and the local Ollama model, including adversarial ideas like "instantly win". Results were validated inside the game.
- **Server checks:** origin checks, rate limit, body limit and graceful errors.

10. Limitations and future work

Known limitations

- Balance was tuned with simulations rather than many human players; late cities are easier once the party reaches the level cap of 50.
- Small local models produce plainer or occasionally odd text.
- Saves live in one browser's `localStorage` only.

Ideas for the future

- AI-generated side quests and trainer banter, still validated by game rules.
- Streaming replies for faster-feeling dialogue.
- Cloud saves and export/import of save files.
- A post-game challenge mode above the level cap.
- Caching AI lore per creature to save requests.